Searching Algo in Vector(HNSW) DB -2

Que.) If we don’t create hierarchy in HNSW, then kya hoga?

HNSW collapses into a slow graph search and you lose its biggest superpower — **speed at scale**.

Let’s break it down properly 👇

1.What hierarchy actually does in HNSW (intuition)

HNSW = **Hierarchical Navigable Small World**

Hierarchy means:

- **Top layers** → very few points, very sparse
- **Bottom layer** → all points, very dense

Search flow:

Top layer (coarse search)

↓

Middle layers (refinement)

↓Bottom layer (exact-ish neighbors)

This is like:

- First finding **which area of the city**
- Then **which street**
- Then **which house**

2.If you remove hierarchy completely

You’re left with:

- Only **one big graph**
- All vectors at same level
- No coarse-to-fine narrowing

So search becomes:

Start from some node → hop neighbors → hop neighbors → hop neighbors…

That means:

Problems

1. Search becomes slow

- You may need **many hops**
- Graph traversal becomes almost linear in worst cases
- Similar to greedy BFS/DFS on a huge graph

2. Bad scalability

- Works okay for **10k vectors**
- Falls apart for **1M+ vectors** (Blinkit / Amazon scale 💀)

3. Local minima problem

- You can get **stuck in a local cluster**
- No higher-level "global view" to escape

Hierarchy helps you *jump far*, then refine.

3.What does it turn into without hierarchy?

Basically it becomes:

Structure	What it resembles
Single-layer HNSW	k-NN graph
No hierarchy	Greedy graph search
Result	Slow + unstable recall

So:

HNSW without hierarchy ≈ graph ANN, not HNSW

4.Real-world analogy (important)

Imagine Google Maps without zoom levels

- You open map → **all roads visible**
- You want to go from Delhi to Mumbai
- You start checking roads **one by one**

Hierarchy = zoom levels

Without it = chaos

5.Why hierarchy is the “secret sauce”

Hierarchy gives:

- **Logarithmic-like search**
- Global navigation
- Balance between speed & accuracy

That’s why:

- FAISS HNSW
- Milvus HNSW
- Weaviate HNSW

All **require hierarchy**

6.One-line takeaway

Without hierarchy, HNSW loses fast global navigation and degrades into slow graph traversal, making it unsuitable for large-scale vector search.

Without hierarchy, you **can get stuck in a loop / local region**

What actually happens

If you only have **one flat graph**:

- Search starts from an entry node
- You move greedily to the “closest” neighbor
- At some point:
  - All neighbors are **worse than current**
  - But **better results exist far away**

Now you’re stuck.

This is called **local minimum trapping**.

Why loops / traps happen in flat graphs

1.Greedy search limitation

Most ANN graph searches are **greedy**:

“Move only if distance decreases”

So:

- No uphill moves allowed
- No global jump mechanism

Result:

- You stop even though better nodes exist elsewhere

2.Dense local clusters

Real vector data is **clustered**:

- Similar products form tight groups
- Each cluster is internally well connected
- But inter-cluster links are sparse

So you get:

ClusterA ← you are here

ClusterB ← actual best result

Flat graph search:

Never escapes Cluster A

3.Cyclic neighbor paths

Since graphs are not trees:

A →B → C →A

If your search algorithm:

- Revisits nodes
- Or explores limited neighbors

You can:

- Loop
- Or keep oscillating near same region

How hierarchy prevents this exact problem

Upper layers = global vision

Top layers:

- Have **long-range edges**

- Connect far-apart regions

So search flow becomes:

Top layer:Jumpacrossclusters↓Lower layer:Refinelocally

Instead of:

“Try to escape cluster blindly”

It becomes:

“Start near correct region, then zoom in”

Important subtle point (exam + interview gold)

Hierarchy does NOT guarantee perfect recall

It dramatically **reduces probability of local trapping**

That’s why:

- HNSW is *approximate*
- But recall is very high in practice

One clean summary

Problem in flat graph ANN

- Greedy search can get stuck in **local minima**
- Loops or dense clusters prevent reaching distant better neighbors
- Best matches may lie in disconnected or weakly connected regions

HNSW solution

- Upper layers provide **global shortcuts**
- Lower layers handle **local precision**
- Hierarchy prevents loop-trapping and local optima

Without hierarchy, ANN graph search can get trapped in local clusters or loops, missing better neighbors located in distant regions of the graph.

1.How Search Begins in HNSW

Entry Point Selection

- HNSW maintains a **single entry point**:
  - A node with the **maximum layer level**
  - Usually one of the earliest inserted nodes

Why a Single Entry Point Works

- Upper layers are **sparse**
- Long-range connections allow fast global movement
- Even a random entry point converges quickly

Initialization Steps

1. Query vector **Q** arrives
2. Start at:
    - Entry point node
    - Highest layer Lmax

2.Search Strategy at Each Layer

At every layer, HNSW performs a **greedy search**:

Greedy Rule

Move to a neighbor **only if** it is closer to the query than the current node.

What This Means

- Only local neighborhood is explored
- No backtracking
- Very fast

3.When Do We Move Down a Layer?

Descent Condition (Key Rule)

You move **down one layer** when:

No neighbor in the current layer is closer to the query than the current node.

In short:

- Local minimum reached **at this layer**
- Best possible position at this resolution

4.Why This Rule Makes Sense

Think in terms of **zoom levels**:

Layer	Purpose
Upper	Find correct region
Middle	Narrow search
Bottom	Exact neighborhood

Once improvement stops:

- Further searching at same layer is useless
- Lower layer offers **more detailed graph**

5.Formal Step-by-Step Search Flow

1. Start at entry point at top layer
2. While improvement exists:
    - Move to closer neighbor
3. When stuck:
    - Descend one layer
    - Continue from same node
4. Repeat until layer 0 is reached

How Does a Node Go UP in HNSW? (Layer Assignment Logic)

Overview

In HNSW, **not all nodes exist in all layers**.

- Every node **must exist at layer 0** (bottom)
- Higher layers contain **fewer and fewer nodes**
- Which layers a node belongs to is decided **at insertion time**

This decision is **randomized but controlled**, not based on distance or importance.

1.When Does This Happen?

Layer assignment happens **when a new node is inserted** into the HNSW index.

Steps at a high level:

1. New vector arrives
2. A **maximum layer level** is sampled
3. Node is inserted from that level down to layer 0

2.Core Idea: Probabilistic Layer Selection

HNSW uses a **randomized level generation** strategy inspired by **skip lists**.

Key Principle

Higher layers should be exponentially sparser.

So:

- Many nodes at layer 0
- Fewer at layer 1
- Very few at layer 2, 3, ...

3.How the Level Is Actually Chosen

Each node gets a **maximum level l** based on a probability distribution.

Formula (Conceptual)

$P(\text{level} \geq l) \approx \exp(-l / m)$

Where:

- l = layer number
- m = level multiplier (controls sparsity)

This means:

- Probability drops exponentially as level increases

4.Intuitive Coin-Flip Explanation

Think like this:

- Every new node:
  - Automatically goes to **layer 0**
  - Then keeps flipping a biased coin:
    - If HEADS → go one layer up
    - If TAILS → stop

Result:

- Most nodes stop early
- Very few reach high layers

This creates a **pyramid structure**.

5.Important Clarification (Important)

Nodes are **NOT promoted** later

Nodes do **NOT climb up dynamically**

Layer is fixed **once at insertion**

No rebalancing like trees

Randomness gives balance naturally

6.Why Random Selection Works

Even though it feels risky, randomness ensures:

- Uniform coverage of space
- No bias toward early or late data
- High probability that **any region has entry points at higher layers**

This avoids:

- Hotspots
- Overcrowded upper layers
- Structural bias

7.What Happens After Level Is Chosen?

Suppose a node gets maxLevel = 3.

Then:

- Node is inserted into:
  - Layer 3
  - Layer 2
  - Layer 1
  - Layer 0

Connection Rule per Layer

At each layer:

- Search neighbors using current graph
- Connect to best M neighbors
- Lower layers → higher M (denser)

ANN algorithms are simply divided into three categories.

How Is the Maximum Level of a Node Decided in REAL HNSW?

Short Truth (no sugarcoating)

- There is **no literal head/tail coin flip loop** in production code
- There **is a mathematical random process**
- Coin-flip explanation is only an **intuition model**

Real implementations use **random numbers + logarithms**.

1.The Actual Formula Used in Practice

In real HNSW implementations (FAISS, hnswlib, Milvus):

level = ⌊ -ln(U) × m ⌋

Where:

- U = uniform random number in (0, 1]
- ln = natural logarithm
- m = level multiplier (typically 1 / ln(M))

This directly produces an **exponentially decaying distribution**.

2.Why This Matches P(level ≥ l) ≈ exp(-l / m)

Let's connect math with intuition:

- ln(U) follows an **exponential distribution**
- Multiplying by m controls how fast probability decays
- Flooring gives an integer level

So:

P(level ≥ l) = exp(-l / m)

This is **not conceptual** — it's mathematically guaranteed.

3.So Is Head/Tail Used or Not?

Not Literally

There is:

- No while loop flipping coins
- No repeated randomness

But Conceptually Equivalent

Coin-flip explanation = **mental model**

Log-based formula = **efficient, deterministic sampling**

They produce the **same statistical distribution**.

4.BIG DOUBT: Why Don't We End Up With Only One Layer?

Your fear is logical 👇

"If tail comes for many nodes, won't all nodes stay at layer 0 → flat graph?"

Here's the key insight

Random ≠ unlucky every time

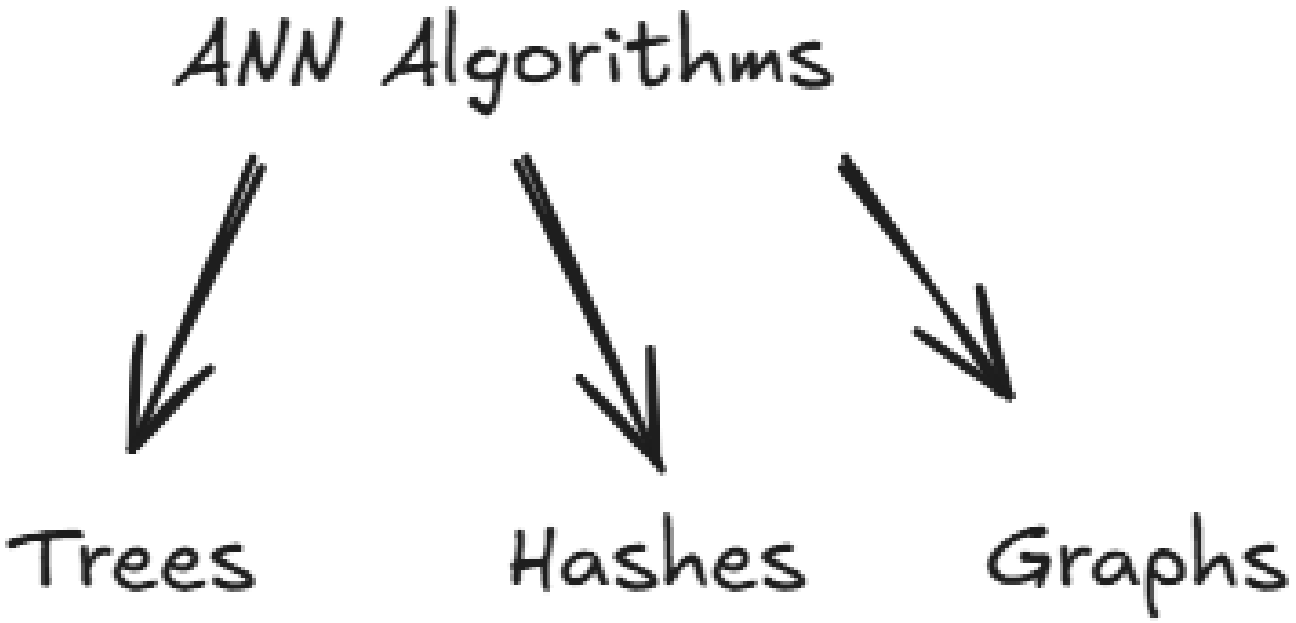
With exponential distribution:

Layer	% of nodes (approx)
0	100%
1	~50%
2	~25%
3	~12%
4	~6%
5	~3%

So:

- Probability **never becomes zero**
- With large N, higher layers **always exist**

Law of large numbers saves us



HNSW slots into the *graph* category. More specifically, it is a *proximity graph*, in which two vertices are linked based on their proximity (closer vertices are linked) — often defined in Euclidean distance.

Probability skip lists -

Skip List = Linked List + Express Lanes (fast like binary search)

Imagine this real-life example

Think of a **long road with many houses**, house numbers are sorted:

1 → 3 → 5 → 7 → 9 → 11 → 13 → 15 → ...

This is a **normal linked list**.

Problem?

To find **13**, you must walk:

1 → 3 → 5 → 7 → 9 → 11 → 13

slow...

Now add “skip roads” (this is Skip List)

We build **multiple levels**:

Top level (express lane – big jumps)

1--------→ 9 --------→ 15

Middle level (medium jumps)

1-----→ 5 -----→ 9 -----→ 13 -----→ 15

Bottom level (normal linked list)

1 → 3 → 5 → 7 → 9 → 11 → 13 → 15

Each level is a **linked list**, just with different jump sizes.

How searching works (step-by-step)

Let's search for **13**

Step 1: Start at **top level**

- Start at 1
- Jump to 9 (still < 13)
- Next jump is 15 (> 13)

Oops, overshoot!

Go down one level, stay at 9

Step 2: Middle level

From 9:

- Jump to 13

Found it

Why this is fast?

Because:

- Top level = **big skips**
- Lower levels = **smaller skips**
- Final level = **exact match**

Similar to **binary search**, but:

- Uses **linked lists**
- No shifting needed for insert/delete

Why “probability” skip list?

When inserting a new element:

- Coin toss decides:
  - Should it appear in upper levels or not?

Example:

- Heads → goes to higher level
- Tails → stops

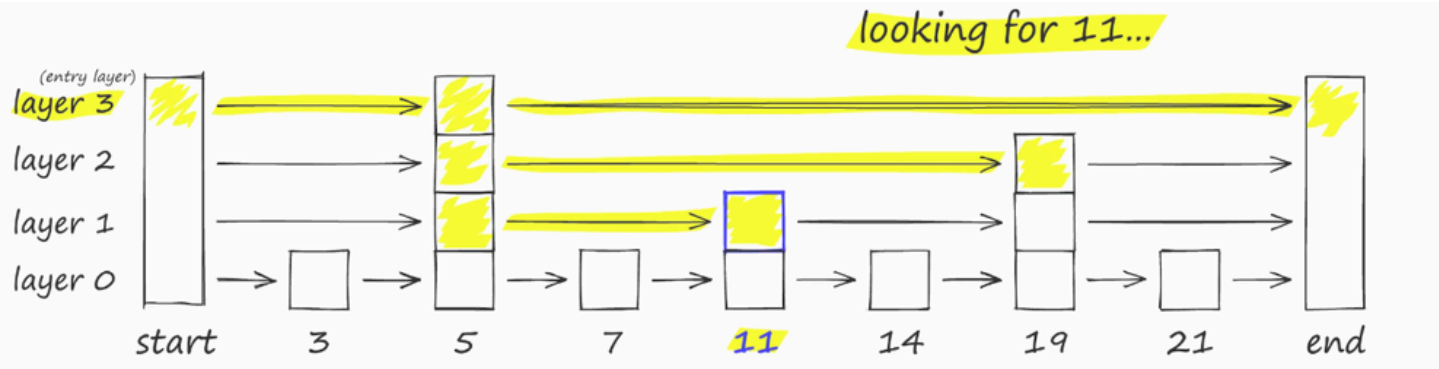
This randomness keeps structure **balanced on average**.

Easy memory trick

Start fast (top), slow down gradually (go down), stop when found

Final ultra-simple summary

- Skip List = many linked lists stacked
- Top = fast jumps
- Bottom = normal list
- Search = move right → overshoot → go down
- Fast search + easy insertion = win



A probability skip list structure, we start on the top layer. If our current key is greater than the key we are searching for (or we reach end), we drop to the next layer.

HNSW inherits the same layered format with longer edges in the highest layers (for fast search) and shorter edges in the lower layers (for accurate search).

Navigable Small World Graphs

Vector search using *Navigable Small World* (NSW) graphs was introduced over the course of several papers from 2011-14 [4, 5, 6]. The idea is that if we take a proximity graph but build it so that we have both long-range and short-range links, then search times are reduced to (poly/)logarithmic complexity.

Each vertex in the graph connects to several other vertices. We call these connected vertices *friends*, and each vertex keeps a *friend list*, creating our graph.

When searching an NSW graph, we begin at a pre-defined *entry-point*. This entry point connects to several nearby vertices. We identify which of these vertices is the closest to our query vector and move there.